```
  def expiration_date_cannot_be_in_the_past
    if expiration_date.present? && expiration_date < Date.today
      errors.add(:expiration_date, "can't be in the past")
    end
  end

  def discount_cannot_be_greater_than_total_value
    if discount > total_value
      errors.add(:discount, "can't be greater than total value")
    end
  end
end
```

By default such validations will run every time you call `valid?`. It is also possible to control when to run these custom validations by giving an `:on` option to the `validate` method, with either: `:create` or `:update`.

```
class Invoice < ActiveRecord::Base
  validate :active_customer, on: :create

  def active_customer
    errors.add(:customer_id, "is not active") unless customer.active?
  end
end
```

7 Working with Validation Errors

In addition to the `valid?` and `invalid?` methods covered earlier, Rails provides a number of methods for working with the `errors` collection and inquiring about the validity of objects.
The following is a list of the most commonly used methods. Please refer to the `ActiveModel::Errors` documentation for a list of all the available methods.

7.1 `errors`

Returns an instance of the class `ActiveModel::Errors` containing all errors. Each key is the attribute name and the value is an array of strings with all errors.

```
class Person < ActiveRecord::Base
  validates :name, presence: true, length: { minimum: 3 }
end

person = Person.new
person.valid? # => false
person.errors.messages
 # => {:name=>["can't be blank", "is too short (minimum is 3
characters)"]}

person = Person.new(name: "John Doe")
person.valid? # => true
person.errors.messages # => {}
```

7.2 `errors[]`

`errors[]` is used when you want to check the error messages for a specific attribute. It returns an array of strings with all error messages for the given attribute, each string with one error message. If there are no errors related to the attribute, it returns an empty array.

```
class Person < ActiveRecord::Base
  validates :name, presence: true, length: { minimum: 3 }
end

person = Person.new(name: "John Doe")
person.valid? # => true
person.errors[:name] # => []
```